# OSLAB – EXPERIMENT – 1

## PROGRAM-1:

```
sainath@Sainath:~$ cd ~/OSLAB
sainath@Sainath:~/OSLAB$ cd fork
sainath@Sainath:~/OSLAB/fork$ cat program1.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 printf("PID = %d\n", getpid());
 return 0;
}
sainath@Sainath:~/OSLAB/fork$ gcc program1.c -o program1
sainath@Sainath:~/OSLAB/fork$ ./program1
PID = 618
```

## PROGRAM-2:

```
sainath@Sainath:~/OSLAB/fork$ cat program2.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 fork();
 printf("Hello\n");
 return 0;
}

sainath@Sainath:~/OSLAB/fork$ gcc program2.c -o program2
sainath@Sainath:~/OSLAB/fork$ ./program2
Hello
Hello
```

# PROGRAM-3:

```
sainath@Sainath:~/OSLAB/fork$ cat program3.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 pid_t pid;
 pid = fork();
 if(pid == 0)
 printf("I am Child\n");
 else
 printf("I am Parent\n");
 return 0;
}

sainath@Sainath:~/OSLAB/fork$ gcc program3.c -o program3
sainath@Sainath:~/OSLAB/fork$ ./program3
I am Parent
I am Child
```

# PROGRAM-4:

```
sainath@Sainath:~/OSLAB/fork$ cat program4.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 pid_t pid;
 pid = fork();
 if(pid == 0)
 {
 printf("Child PID = %d\n", getpid());
 printf("Parent PID = %d\n", getppid());
 }
 else
 {
 printf("Parent PID = %d\n", getpid());
 printf("Child PID = %d\n", pid);
 }
 return 0;
}
sainath@Sainath:~/OSLAB/fork$ gcc program4.c -o program4
sainath@Sainath:~/OSLAB/fork$ ./program4
Parent PID = 718
Child PID = 719
Child PID = 719
Parent PID = 628
```

# PROGRAM-5:

```
sainath@Sainath:~/OSLAB/fork$ cat program5.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
 pid_t pid = fork();
 if(pid == 0)
 {
 printf("Child Executing\n");
 }
 else
 {
 wait(NULL);
 printf("Parent Executing\n");
 }
 return 0;
}

sainath@Sainath:~/OSLAB/fork$ gcc program5.c -o program5
sainath@Sainath:~/OSLAB/fork$ ./program5
Child Executing
Parent Executing
```

# PROGRAM-6:

```
sainath@Sainath:~/OSLAB/fork$ cat program6.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 fork();
 fork();
 printf("PID = %d\n", getpid());
 return 0;
}
sainath@Sainath:~/OSLAB/fork$ gcc program6.c -o program6
sainath@Sainath:~/OSLAB/fork$ ./program6
PID = 735
PID = 737
PID = 736
PID = 738
```

## PROGRAM-7:

```
sainath@Sainath:~/OSLAB/fork$ cat program7.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 pid_t pid = fork();
 if(pid == 0)
 {
 printf("Child executing ls...\n");
 execl("/bin/ls","ls","-l",NULL);
 }
 else
 {
 printf("Parent Waiting...\n");
 }
 return 0;
}
sainath@Sainath:~/OSLAB/fork$ gcc program7.c -o program7
sainath@Sainath:~/OSLAB/fork$ ./program7
Parent Waiting...
Child executing ls...
sainath@Sainath:~/OSLAB/fork$ total 168
-rwxr-xr-x 1 sainath sainath 16400 Jul 29 03:49 process
-rwxr-xr-x 1 sainath sainath 16008 Jul 31 19:38 program1
-rw-r--r-- 1 sainath sainath    98 Jul 29 03:33 program1.c
-rw-r--r-- 1 sainath sainath   132 Jul 29 03:46 program10.c
-rwxr-xr-x 1 sainath sainath 16008 Jul 31 19:39 program2
-rw-r--r-- 1 sainath sainath    95 Jul 29 03:38 program2.c
-rwxr-xr-x 1 sainath sainath 16008 Jul 31 19:40 program3
-rw-r--r-- 1 sainath sainath   164 Jul 29 03:39 program3.c
-rwxr-xr-x 1 sainath sainath 16096 Jul 31 19:40 program4
-rw-r--r-- 1 sainath sainath   278 Jul 29 03:41 program4.c
-rwxr-xr-x 1 sainath sainath 16048 Jul 31 19:41 program5
-rw-r--r-- 1 sainath sainath   215 Jul 29 03:43 program5.c
-rwxr-xr-x 1 sainath sainath 16048 Jul 31 19:41 program6
-rw-r--r-- 1 sainath sainath   116 Jul 29 03:44 program6.c
-rwxr-xr-x 1 sainath sainath 16048 Jul 31 19:42 program7
-rw-r--r-- 1 sainath sainath   220 Jul 29 03:45 program7.c
-rw-r--r-- 1 sainath sainath   629 Jul 29 03:48 program9.c
```

# PROGRAM-9:

```
sainath@Sainath:~/OSLAB/fork$ cat program9.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
 char cmd[50];
 printf("Enter Linux Command: ");
 scanf("%s", cmd);
 pid_t pid = fork();
 if(pid < 0)
 {
 printf("Fork Failed\n");
 return 1;
 }
 else if(pid == 0)
 {
 printf("\nChild Process\n");
 printf("Child PID : %d\n", getpid());
 printf("Parent PID: %d\n", getppid());
 execlp(cmd, cmd, NULL);
 perror("Command Execution Failed");
 exit(1);
 }
 else
 {
 printf("\nParent Process\n");
 printf("Parent PID : %d\n", getpid());
 printf("Child PID : %d\n", pid);
 wait(NULL);
 printf("\nChild Process Completed\n");
 }
 return 0;
}

sainath@Sainath:~/OSLAB/fork$ gcc program9.c -o program9
sainath@Sainath:~/OSLAB/fork$ ./program9
Enter Linux Command: ls

Parent Process
Parent PID : 761
Child PID : 766

Child Process
Child PID : 766
Parent PID: 761
process   program1.c   program2    program3    program4    program5    program6    program7    program9
program1  program10.c  program2.c  program3.c  program4.c  program5.c  program6.c  program7.c  program9.c
```

# PROGRAM-10:

```
Child Process Completed
sainath@Sainath:~/OSLAB/fork$ cat program10.c
#include <stdio.h>
#include <unistd.h>
int main()
{
 int i;
 for(i=0;i<3;i++)
 fork();
 printf("PID = %d\n",getpid());
 return 0;
}
sainath@Sainath:~/OSLAB/fork$ gcc program10.c -o program10
sainath@Sainath:~/OSLAB/fork$ ./program10
PID = 773
PID = 775
PID = 774
PID = 777
PID = 779
PID = 778
PID = 780
PID = 776
```